



Especialización en Diseño y Desarrollo de Videojuegos
Facultad de ingeniería

ACA 1: Diseñar la arquitectura orientada a objetos de un videojuego tipo Breakout en Lua, modelando entidades, responsabilidades y máquinas de estado en formato UML y pseudocódigo

Presentado por:

Jhoan Andrés Castelblanco Utria

Julian Eleicer Orellanos

Bogotá, Cundinamarca 26 de julio de 2026

Repositorio Github: <https://github.com/johancastel/Breakout-Guardians-of-the-Forge>

1. Diagrama UML de clases del Breakout.
- 2.
3. ANÁLISIS Y RAZONAMIENTO DE LAS ENTIDADES
4. Análisis de Entidades
- 5.
6. Breakout: Guardians of the Forge
- 7.
8. Clase: Game
- 9.
10. Responsabilidad
- 11.
12. Administrar el flujo general del videojuego, controlar los estados del juego, iniciar partidas, gestionar niveles y coordinar la comunicación entre todas las entidades.
- 13.
14. Atributos
- 15.
16. - currentState
17. - currentLevel
18. - score
19. - lives
20. - isRunning
- 21.
22. Métodos
- 23.
24. - load()
25. - update(dt)
26. - draw()
27. - changeState()
28. - restartGame()
29. - nextLevel()

- 30.
- 31. Colabora con
- 32.
- 33. - Level
- 34. - UI
- 35. - Ball
- 36. - Paddle
- 37.
- 38.
- 39.
- 40. Clase: Ball
- 41.
- 42. Responsabilidad
- 43.
- 44. Representar la bola principal (Energy Core), controlar su movimiento, detectar colisiones y aplicar efectos especiales obtenidos mediante poderes.
- 45.
- 46. Atributos
- 47.
- 48. - Ejex
- 49. - Ejey
- 50. - radius
- 51. - speedX
- 52. - speedY
- 53. - damage
- 54.
- 55. MÃ©todos
- 56.
- 57. - update(dt)
- 58. - draw()
- 59. - move()

- 60. - bounce()
- 61. - reset()
- 62. - applyPower()
- 63.
- 64. Colabora con
- 65.
- 66. - Paddle
- 67. - Brick
- 68. - Game
- 69. - PowerUp
- 70.
- 71.
- 72.
- 73. Clase: Paddle
- 74.
- 75. Responsabilidad
- 76.
- 77. Representar el Forge Shield controlado por el jugador, permitiendo dirigir la bola y recibir los efectos de los poderes.
- 78.
- 79. Atributos
- 80.
- 81. - x
- 82. - width
- 83. - height
- 84. - speed
- 85. - currentPower
- 86.
- 87. MÃ©todos
- 88.
- 89. - moveLeft()

90. - moveRight()
91. - update(dt)
92. - draw()
93. - catchPower()
94.
95. Colabora con
96.
97. - Ball
98. - PowerUp
99.
100.
101.
102. Clase: Brick
103.
104. Responsabilidad
105.
106. Representar los bloques del nivel, administrar su resistencia, almacenar poderes y
 desaparecer cuando su resistencia llegue a cero.
107.
108. Atributos
109.
110. - x
111. - y
112. - width
113. - height
114. - health
115. - type
116. - containsPower
117.
118. MÃ©todos
119.

- 120. - draw()
- 121. - hit()
- 122. - destroy()
- 123. - releasePower()
- 124.
- 125. Colabora con
- 126.
- 127. - Ball
- 128. - PowerUp
- 129.
- 130.
- 131.
- 132. Clase: Level
- 133.
- 134. Responsabilidad
- 135.
- 136. Administrar la distribución de los bloques, controlar el progreso del nivel y verificar cuándo el jugador completa la cámara actual.
- 137.
- 138. Atributos
- 139.
- 140. - levelNumber
- 141. - bricks
- 142. - background
- 143. - difficulty
- 144.
- 145. Métodos
- 146.
- 147. - loadLevel()
- 148. - update()
- 149. - draw()

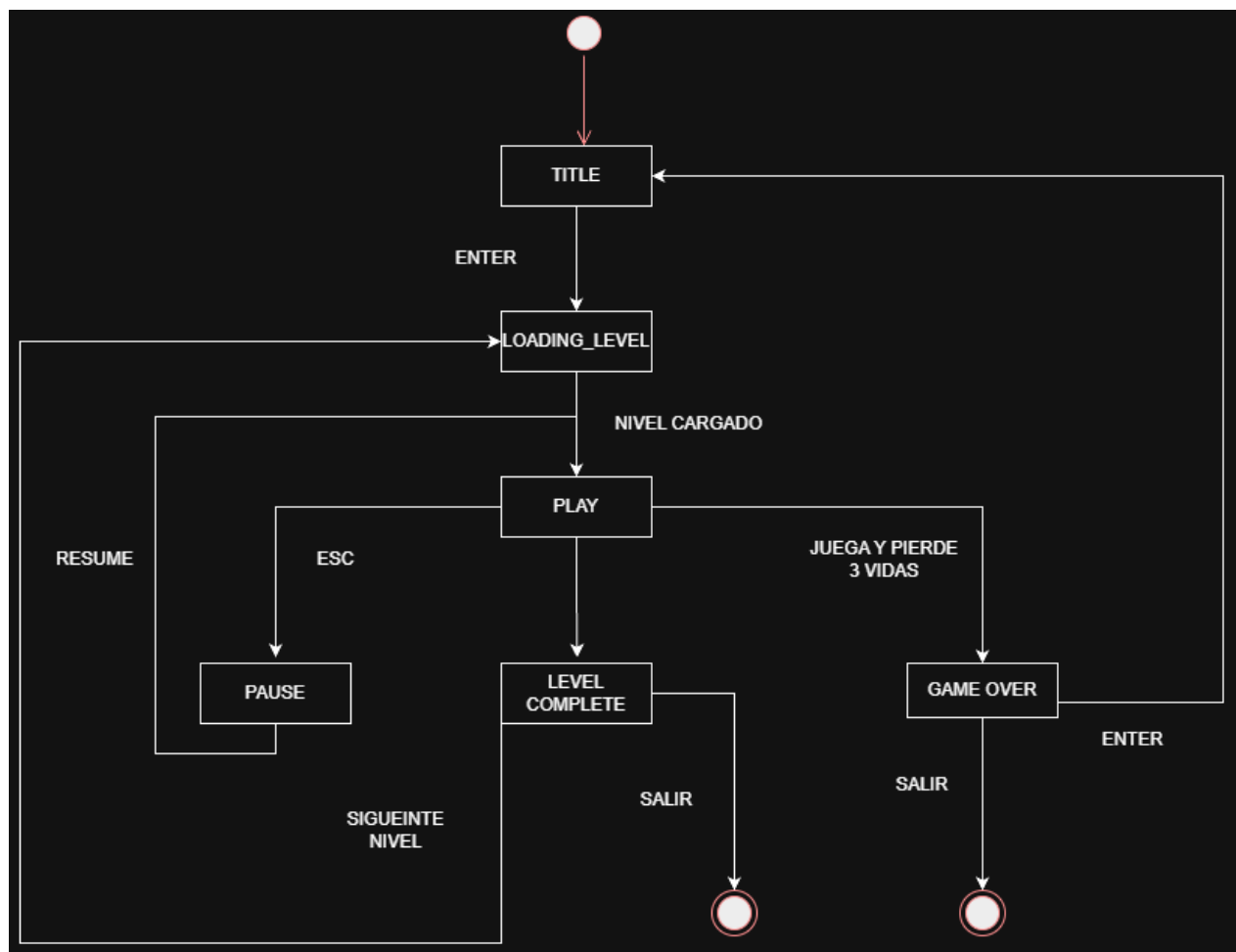
150. - isCompleted()
151.
152. Colabora con
153.
154. - Brick
155. - Game
156.
157.
158.
159. Clase: UI
160.
161. Responsabilidad
162.
163. Mostrar toda la informaci3n del jugador durante la partida, incluyendo vidas,
 puntaje, nivel y mensajes del juego.
164.
165. Atributos
166.
167. - score
168. - lives
169. - currentLevel
170. - message
171.
172. M3todos
173.
174. - drawHUD()
175. - updateScore()
176. - updateLives()
177. - showMessage()
178.
179. Colabora con

180.
181. - Game
182.
183.
184.
185. Clase: PowerUp
186.
187. Responsabilidad
188.
189. Administrar los poderes especiales que aparecen al destruir determinados bloques y aplicar sus efectos sobre la barra o la bola.
190.
191. Atributos
192.
193. - type
194. - duration
195. - speed
196. - isActive
197.
198. MÃ©todos
199.
200. - spawn()
201. - update(dt)
202. - draw()
203. - apply()
204. - remove()
205.
206. Colabora con
207.
208. - Ball
209. - Paddle

210. - Brick
211. - Game
- 212.
- 213.



214. Diagrama de la máquina de estados del juego.



215. Tabla de responsabilidades por clase (clase / atributos / métodos / colaboradores).

table of responsible part - Breakout: Guardians of the Forge				
Clase	Responsabilidad	Atributos	Métodos	Colaboradores
Game	Administrar el flujo general del videojuego, controlar los estados, iniciar partidas y coordinar las entidades principales.	currentState, score, lives	load(), update(), draw(), restartGame()	Level, UI, Ball
Ball	Representar la bola principal (Energy Core), controlar su movimiento, detectar colisiones y aplicar efectos especiales.	x, y, radius, speedX, speedY	move(), bounce(), draw()	Paddle, Brick, PowerUp
Paddle	Representar el Forge Shield controlado por el jugador y permitir el desplazamiento horizontal para dirigir la bola.	x, y, width, speed	moveLeft(), moveRight(), catchPower()	Ball, PowerUp
Brick	Representar los bloques del nivel, administrar su resistencia y liberar poderes al ser destruidos.	health, type, containsPower	hit(), destroy(), releasePower()	Ball
Level	Gestionar la información de cada nivel, cargar los bloques y verificar cuándo un nivel ha sido completado.	levelNumber, bricks, difficulty	loadLevel(), update(), isCompleted()	Brick, Game
UI	Mostrar la interfaz gráfica del juego, incluyendo puntaje, vidas, nivel actual y mensajes al jugador.	score, lives, currentLevel	drawHUD(), updateScore()	Game
PowerUp	Administrar los poderes especiales que modifican temporalmente el comportamiento de la bola o la barra.	type, duration, isActive	apply(), remove(), update()	Ball, Paddle, Brick

216. Pseudocódigo del Game Loop principal.

Algoritmo General

INICIO

Inicializar el juego

Cargar recursos del juego

- Sprites
- Sonidos
- Fuentes
- Configuración

Establecer el estado inicial como TITLE

Mientras la aplicación esté en ejecución hacer

Leer la entrada del jugador

Según el estado actual del juego hacer

TITLE

Mostrar pantalla principal

Si el jugador presiona ENTER entonces

Cambiar estado a LOADING_LEVEL

Fin Si

LOADING_LEVEL

Cargar el nivel actual

Crear los bloques

Inicializar la pelota

Posicionar la barra

Inicializar la interfaz

Cambiar estado a PLAY

PLAY

Actualizar la barra

Actualizar la pelota

Detectar colisiones

Actualizar bloques

Actualizar poderes

Actualizar la interfaz

Dibujar todos los elementos del juego

Si el jugador presiona ESC entonces

Cambiar estado a PAUSE

Fin Si

Si el jugador pierde todas las vidas entonces

Cambiar estado a GAME_OVER

Fin Si

Si todos los bloques fueron destruidos entonces

Cambiar estado a LEVEL_COMPLETE

Fin Si

PAUSE

Mostrar menú de pausa

Esperar la decisión del jugador

Si el jugador selecciona CONTINUAR entonces

Cambiar estado a PLAY

Fin Si

Si el jugador selecciona SALIR entonces

Finalizar aplicación

Fin Si

LEVEL_COMPLETE

Mostrar mensaje de nivel completado

Calcular bonificación

Incrementar el número del nivel

Si existen más niveles entonces

Cambiar estado a LOADING_LEVEL

En caso contrario

Cambiar estado a TITLE

Fin Si

GAME_OVER

Mostrar pantalla de Game Over

Mostrar puntaje final

Si el jugador presiona ENTER entonces

Reiniciar partida

Cambiar estado a TITLE

Fin Si

Si el jugador selecciona SALIR entonces

Finalizar aplicación

Fin Si

Fin Según

Fin Mientras

FIN

217. Reflexión

Hacer este ACA realmente sÃ- nos pareciÃ³ muy fascinante ya que no lo tomamos como solo un entregable sino por el contrario como un producto que debe tener inicio y debe tener fin, claramente no se me está solicitando hacer un juego, ¡pero!, si buscamos buenas prácticas de creación, estructura y organización para la creación de un video juego, claro, falta mucho como bases de datos GDD entre muchas cosas, pero quería estructurar un proyecto que tenga una vista seria y responsable.

Ya en materia de aprendizaje comprendimos que el diseño de un videojuego comienza mucho antes de escribir código. Al iniciar el trabajo tenÃ-amos varias dudas sobre los entregables solicitados, especialmente porque confundíamos el diagrama UML con el pseudocódigo. A medida que desarrollamos la actividad entendimos que el UML permite representar la estructura del videojuego mediante clases, atributos, métodos y relaciones, mientras que el pseudocódigo describe la lógica del funcionamiento del juego y del Game Loop sin depender de un lenguaje de programación.

Otro aspecto nuevo para nosotros fue la tabla de responsabilidades por clase. Antes de esta actividad no conocíamos este tipo de documento ni su importancia dentro del diseño orientado a objetos. Al elaborarla comprendimos que ayuda a definir claramente la función de cada clase, facilitando la organización del proyecto y la creación del UML y evitando mezclar responsabilidades entre las diferentes entidades del videojuego.

Otro aprendizaje importante fue volver a utilizar archivos con formato **Markdown (.md)** para documentar el proyecto. Aunque ya conocíamos este formato, hacía bastante tiempo no lo utilizábamos y esta actividad permitió recordar nuevamente su sintaxis y las ventajas que ofrece para organizar documentación técnica dentro de un proyecto de software. Gracias al uso de Markdown fue posible estructurar el concepto del juego, las entidades y el pseudocódigo de una manera clara, ordenada y fácil de mantener, tal como se realiza en muchos proyectos reales de desarrollo de software.

Adicionalmente, esta actividad permitió tener un primer acercamiento a **Lua** y al motor **Love2D**. Aunque todavía no desarrollamos el videojuego como tal, fue interesante comprender la estructura básica que debe tener un proyecto, conocer la función de archivos como `main.lua` y `conf.lua`, así como entender que Love2D utiliza funciones específicas como `love.load()`, `love.update()` y `love.draw()` para controlar el ciclo principal del juego. Este primer contacto con la sintaxis de Lua despertó aún más nuestro interés por continuar aprendiendo el lenguaje durante el desarrollo de la especialización.

¿Queremos cambiar algunos de nuestro diseño?

Desde una perspectiva autocrítica y con el fin de seguir mejores prácticas en ingeniería de software, si tuviéramos que rediseñar el proyecto, realizaríamos los siguientes cambios:

1. Estructura de Poderes (PowerUps): En el diseño actual, los poderes se identifican a través de un atributo 'type' (de tipo string). Para seguir el principio Abierto/Cerrado (Open/Closed Principle) de SOLID, implementaremos una jerarquía de clases donde cada poder sea una subclase de una clase base 'PowerUp'. Esto evitará estructuras condicionales complejas a medida que se añadan más tipos de poderes.
2. Desacoplamiento del Sistema de Física: Actualmente la lógica de rebotes y colisiones está distribuida directamente en 'Ball', 'Paddle' y 'Brick'. Consideramos que será mucho más limpio y modular implementar un 'CollisionManager' o un sistema de física dedicado, reduciendo la dependencia directa entre estas entidades y facilitando futuras modificaciones en el comportamiento de colisión.
3. Modularidad en la UI: Separaremos de forma más estricta la representación de datos del juego del renderizado visual de la interfaz. Esto facilitará implementar múltiples pieles (skins) o cambiar la resolución del juego sin afectar la lógica interna.